

CSC1029 OO Programming

Lecture 12 - Conventions





What's to come in today's lecture?

- Java Naming Conventions
 - Guidelines for Comments
 - What makes a 'good' class?
 - Designing Classes & Methods
 - Method Overloading (Compile-time Polymorphism)
- 

Java Naming Conventions ⁽¹⁾

By *convention*, we need some guidelines when choosing identifier names for class, package, variable, constant, method, etc.

Conventions are defined by most major companies.

Following these *conventions* can vastly improve code readability.

Not enforced by the compiler (Eclipse) – but we will in this course!

Java Naming Conventions (2)

Classes ...

Class names should be *nouns*, in mixed case with the first letter of each internal word capitalised. Use whole words and must avoid acronyms and abbreviations.

```
public class Student{  
    ...  
}  
public class BankAccount{  
    ...  
}
```

Methods ...

Methods should be *verbs*, in mixed case with the first letter lowercase and with the first letter of each internal word capitalised.

```
public void printValue() {  
    ...  
}
```

Java Naming Conventions (3)

Variables ...

- Variable names *should be short* yet meaningful.
- Should *not* start with underscore('_') or dollar sign '\$' characters.
- Should be *mnemonic* i.e, designed to indicate to the casual observer the intent of its use.
- For multiple words use mixed case with the first letter lowercase and with the first letter of each internal word capitalised.

```
int index;  
String firstName;  
double balance;
```

Java Naming Conventions ⁽⁴⁾

Constants ...

- Should be all uppercase with words separated by underscores (“_”).

```
final int MAX_VALUE = 5;
```

Packages ...

- The prefix of a unique package name is always written in all-lowercase ASCII letters and should be one of the top-level domain names, like com, edu, gov, mil, net, org.

```
java.lang  
java.util  
javax.swing
```

Javadoc and Other Comments ⁽¹⁾

An overall comment (Javadoc) for the class.

```
import java.util.Scanner;

/**
 * This class represents ...
 * @author Paul Sage
 * @version V1.0
 *
 */
public class Menu {
```

Javadoc and Other Comments ⁽²⁾

Use single line comments to document variables.

```
public class Menu {  
    private String options[];    // array of strings representing user options  
    private String title;        // menu title  
}
```


Javadoc and Other Comments ⁽³⁾

Javadoc comment should appear above each method – describe the method purpose and parameters.

```
/**
 * Constructor for class
 * @param title - the menu title
 * @param options - the options for user selection
 */
public Menu(String title, String options[]) {
    this.title = title;
    copyOptions(options);
}
```

Eclipse will help – place edit cursor above the ‘p’ for public – type `/**` followed by *return*.

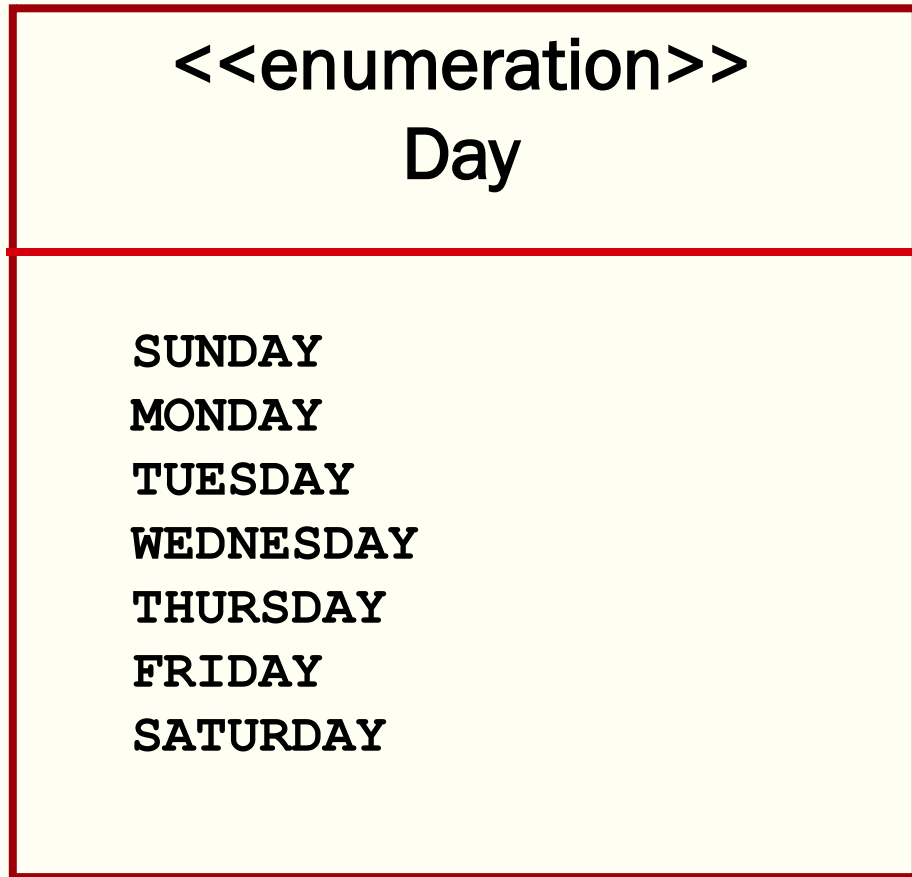


Javadoc and Other Comments ⁽⁴⁾

Javadoc comment should appear above each method – describe any return value.

```
/**
 * Requests and read a user choice
 * @return - the user choice
 */
public int getChoice() {
    Scanner input = new Scanner(System.in);
    System.out.print("Enter choice: ");
    int choice = input.nextInt();
    input.close();
    return choice;
}
```

Enumerated Types in UML



```
enum Day {  
    SUNDAY,  
    MONDAY,  
    TUESDAY,  
    WEDNESDAY,  
    THURSDAY,  
    FRIDAY,  
    SATURDAY;  
}
```

Some General Rules

1. Use *meaningful* identifier names at all times
2. Comments (Javadoc & other) are *mandatory*
3. Variables should be *initialised* – avoid relying on default values
4. Don't leave *unused* variables/methods in your code
5. For classes (other than test programs) *do not* write to the console or use a Scanner to read from the KB – unless the class is specifically designed for this
6. All I/O activities should be restricted to the test program

What Makes a Good Class? ⁽¹⁾

A class should represent a single concept from a problem domain.

Some classes you may have seen before ...

Rectangle : to represent a geometric shape in mathematics

BankAccount : an abstraction (representation) of a real-world concept

The ***state*** and ***behaviour*** of objects generated from these classes is relatively easy to understand:

a rectangle has ***width*** and ***height***

you can ***withdraw*** and ***deposit*** money in a bank

What Makes a Good Class? (2)

Generally – concepts from a *domain related to a programs purpose* make good classes

- The *name* for a class *should be a noun* which describes a *concept*.
- So, *look for the nouns* in the problem *specification*

What Makes a Good Class? (3)

Sometimes a class *will not* be used to create objects ...

Such classes may be defined to contain only a lists of ***static*** constants and methods.

For example, the ***Math class*** is defined in this way – never used to create instances of Math

Such a class is called a ***utility class***.

Finally, you will have seen some classes with only a ***main*** method – their sole purpose is to define the starting point of a program.

Method Design – Overloading ⁽³⁾

In a class definition, it is common to have *more than one* method with the same name.

This is known as *method overloading* and is one of two types of *polymorphic* behaviour in OOP.

Polymorphism – “One name, multiple methods”

Polymorphism – in this case, a *single method name can be reused* (overloaded) a number of times to enable a number of alternative approaches to a single process.

Method overloading is also known as *compile-time Polymorphism*

Method Design – Overloading ⁽⁴⁾

For example, consider the following method calls to *println* ...

```
System.out.println();           // no parameters
System.out.println("Hello World!"); // String parameter
System.out.println(42);         // int parameter
System.out.println(1.4);        // double parameter
```

Polymorphism – “One name, multiple methods”

For method *println*, the same name has been reused (**overloaded**) a number of times to enable a number of different ways of writing information to the console.

This mechanism can be used to enhance code flexibility.

Method Design - Overloading (5)

Method `println` is defined within class `PrintStream` ... take a look for the associated Javadoc.

<code>void</code>	<code>println()</code> Terminates the current line by writing the line separator string.
<code>void</code>	<code>println(boolean x)</code> Prints a boolean and then terminate the line.
<code>void</code>	<code>println(char x)</code> Prints a character and then terminate the line.
<code>void</code>	<code>println(char[] x)</code> Prints an array of characters and then terminate the line.
<code>void</code>	<code>println(double x)</code> Prints a double and then terminate the line.
<code>void</code>	<code>println(float x)</code> Prints a float and then terminate the line.
<code>void</code>	<code>println(int x)</code> Prints an integer and then terminate the line.
<code>void</code>	<code>println(long x)</code> Prints a long and then terminate the line.
<code>void</code>	<code>println(Object x)</code> Prints an Object and then terminate the line.
<code>void</code>	<code>println(String x)</code> Prints a String and then terminate the line.

Method Design – Overloading (6)

The PrintStream class definition looks something like ...

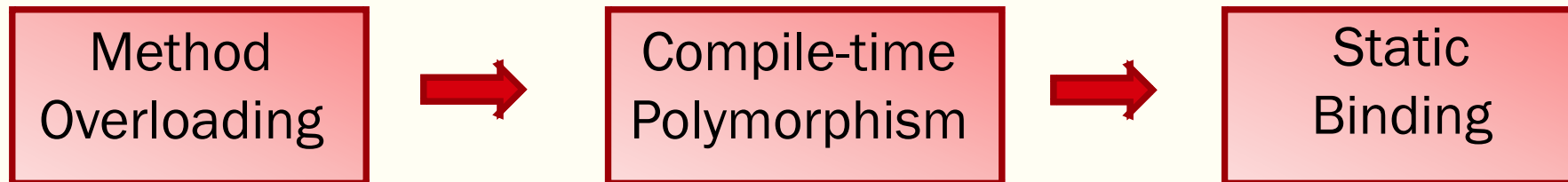
```
public class PrintStream ... {  
    ...  
    public void println() { ... }  
    public void println(String x) { ... }  
    public void println(int x) { ... }  
    public void println(double x) { ... }  
    ...  
}
```

For method overloading to work – method definitions must differ by *number* and/or *type* of parameters.

Method Design – Overloading ⁽⁷⁾

For method overloading to work – method definitions must *differ* by number and/or type of parameters:

- in other words, each *method prototype/heading/signature must be unique*
- the jdk must be able to differentiate between overloaded methods on compilation
- hence the name *compile-time polymorphism*
- a method call is bound (or tied) to its associated method definition at compile time
- this called *static binding*



Method Design – Overloading ⁽⁸⁾

The method *overloading* technique is often used with object *construction*.

Overloading object constructors provides flexibility in object initialisation.

Consider the *BankAccount* example from the previous lecture.

```
public BankAccount(String name) {  
    this.name = name;  
    this.acctNum = nextAcctNum();  
    this.balance = 0.0;  
}  
  
public BankAccount(String name,  
                    double amt) {  
    this.name = name;  
    this.acctNum = nextAcctNum();  
    if (amt > 0.0) {  
        this.balance = amt;  
    }  
    else {  
        this.balance = 0.0;  
    }  
}
```

Designing Good Classes & Methods ⁽¹⁾

Dependencies

Dependencies should be *minimised*.

Again, consider the *BankAccount* class, which defines a statement method.

What if this method directly displayed information on the console?

Where are the dependencies in this case?

```
// this version returns a String
public String statement() {
    String result = "";
    result += "Balance is:" + balance;
    return result;
}

// this version writes to the console
public void statement() {
    String result = "";
    result += "Balance is:" + balance;
    System.out.println(result);
}
```

Designing Good Classes & Methods (2)

Dependencies

The 2nd version of statement *depends on System.out* to work.

This is a *bad* route to take – not all environments have access to the console!

This dependency should be removed – *decoupled* – I/O should take place in a separate class.

```
// this version returns a String
public String statement() {
    String result = "";
    result += "Balance is:" + balance;
    return result;
}

// this version writes to the console
public void statement() {
    String result = "";
    result += "Balance is:" + balance;
    System.out.println(result);
}
```

Designing Good Classes & Methods (3)

Separating Accessors & Mutators

Mutators (setters) *change* the state of an object.

Accessors (getters) asks an object to obtain a value or computed result – *without changing* object state.

Some classes will only have **accessors** – state can't change – *immutable*.

For example, the **String** class.

It's *always safe* to pass around references to *immutable* objects. Not the case for *mutable* objects.



What's Next?

Preparation for Assessment 1